



ONLINE FOOD ORDERING SYSTEM



A PROJECT REPORT

Submitted by

SHOBANA M	2403811720522051
SIVARANJANI S	2403811720522052
SWETHA M	2403811720522055
TAMILAZHAKI	2403811720522056

*in partial fulfillment of the requirements for the award degree of
Bachelor in Technology*

CGB1221-DATABASE MANAGEMENT SYSTEM

**DEPARTMENT OF INFORMATION TECHNOLOGY
K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM - 621112

MAY 2026



ONLINE FOOD ORDERING SYSTEM



A PROJECT REPORT

Submitted by

SHOBANA M	2403811720522051
SIVARANJANI S	2403811720522052
SWETHA M	2403811720522055
TAMILAZHAKI	2403811720522056

*in partial fulfillment of the requirements for the award degree of
Bachelor in Technology*

CGB1221-DATABASE MANAGEMENT SYSTEM

**DEPARTMENT OF INFORMATION TECHNOLOGY
K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM - 621112

MAY 2026

**K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM - 621112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled “**ONLINE FOOD ORDERING SYSTEM**” has been carried out by the students SHOBANA M, SIVARANJINI S, SWETHA M, TAMILAZHAKI A . The work reported here in is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce:

Mr.A.MALARMANNAN
SUPERVISOR
Assistant Professor
Information Technology
K.Ramakrishnan College of
Technology(Autonomous)
Samayapuram-621 112

Dr. C. SHYAMALA
HEAD OF THE DEPARTMENT
Associate Professor
Information Technology
K.Ramakrishnan College of
Technology(Autonomous)
Samayapuram-621 112

INTERNAL EXAMINER

EXTERNAL EXAMNIER

ABSTRACT

An Online Food Ordering System is a web-based application designed to simplify the process of ordering food from restaurants through digital platforms. The system allows users to browse menus, select food items, place orders, and make payments online without visiting the restaurant physically. It provides an efficient and user-friendly interface where customers can register, log in, view available dishes, and track their orders in real time. For restaurant administrators, the system offers features to manage menus, update prices, monitor orders, and maintain customer records. This automation reduces manual errors, saves time, and enhances customer satisfaction by providing quick and convenient services. The system is typically developed using technologies such as HTML, CSS, PHP, and a database like MySQL to store and manage data.

Keywords:

Online Food Ordering, E-Commerce, Web Application, Order Management, Payment Gateway, Database Management, Real-Time Tracking, User Interface, Customer Experience.

ACKNOWLEDGEMENT

We thank our **Dr. N.Vasudevan**, Principal, for his valuable suggestions and support during the course of my research work.

We thank our **Dr.C.Shyamala**, Head of the Department, Associate Professor, Department of Information Technology for his/her valuable suggestions and support during the course of my research work.

We wish to record my deep sense of gratitude and profound thanks to my Guide **Mr.A.Malarmannan**, Assistant Professor ,Information Technology for his keen interest, inspiring guidance, constant encouragement with my work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mr.A.Malarmannan** , Assistant Professor, Information Technology for his/her valuable suggestions and support during the course of my research work.

We also thank the faculty and non-teaching staff members of the Department of Information Technology, K. Ramakrishnan College of Technology(Autonomous),Samayapuram-621112, for their valuable support throughout the course of my research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF FIGURES	vii
	LIST OF SYMBOLS AND ABBREVIATIONS	viii
1	INTRODUCTION	1
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVES OF THE PROJECT	2
	1.4 SCOPE OF THE PROJECT	
	1.5 DATABASE MANAGEMENT SYSTEM CONCEPTS	3
2	LITERATURE SURVEY	4
	2.1 EXISTING SYSTEM	4
	2.2 LIMITATIONS OF EXISTING SYSTEM	5
	2.3 PROPOSED SYSTEM	5
	2.4 ADVANTAGES OF PROPOSED SYSTEM	6
3	SYSTEM DESIGN	7
	3.1 ENTITY RELATIONSHIP (ER) DIAGRAM	7
	3.2 DATABASE DESIGN	8
	3.2.1 Table Structure	9
	3.2.2 Attributes	9
	3.4.3 Keys (Primary & Foreign)	9
4	PROJECT METHODOLOGY	10
	4.1 WORKFLOW OF THE SYSTEM	10

CHAPTER NO.	TITLE	PAGE NO.
	4.2 BLOCK DIAGRAM	11
5	SYSTEM REQUIREMENTS	12
	5.1 HARDWARE REQUIREMENTS	12
	5.2 SOFTWARE REQUIREMENTS	13
6	MODULE DESCRIPTION	14
	6.1 USER MODULE	14
	6.2 ADMIN MODULE	14
	6.3 DATABASE MODULE	15
	6.4 AUTHENTICATION MODULE	15
	6.5 REPORT GENERATION MODULE	15
7	IMPLEMENTATION	16
	7.1 DATABASE CREATION	16
	7.2 SQL QUERIES	17
	7.2.1 Insert	17
	7.2.2 Select	17
	7.2.3 Update	17
	7.2.4 Delete	18
	7.3 CONSTRAINTS	18
	7.3.1 PRIMARY KEY	18
	7.3.2 FOREIGN KEY	18
	7.3.3 NOT NULL	18
	7.3.4 UNIQUE	18
8	CONCLUSION	19
9	APPENDIX A SOURCE CODE	20
	APPENDIX B SCREEN SHOT	36
	REFERENCE	39

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Entity Relationship Model	8
4.1	Online Food Ordering System	11

LIST OF SYMBOLS AND ABBREVIATIONS

DBMS	-	Database Management System
ER	-	Entity Relationship
SQL	-	Structured Query Language
UI	-	User Interface
UX	-	User Experience
HTML	-	HyperText Markup Language
CSS	-	Cascading Style Sheet
JS	-	JavaScript
API	-	Application Programming Interface
URL	-	Uniform Resource Locator
HTTP	-	HyperText Transfer Protocol
HTTPS	-	HyperText Transfer Protocol Secure
PK	-	Primary Key
FK	-	Foreign Key
IDE	-	Integrated Development Environment
DB	-	Database

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The Online Food Ordering System is a web-based application designed to simplify the process of ordering food from restaurants. It provides a digital platform where users can browse menus, select food items, place orders, and make payments conveniently from their devices. This system eliminates the need for traditional phone-based ordering and reduces manual errors.

The application typically includes features such as user registration, login, menu display, cart management, and order tracking. Customers can explore various food categories, view item descriptions, and check prices before placing an order. The system also supports secure payment options and generates order confirmations instantly. On the restaurant side, administrators can manage food items, update prices, track orders, and generate reports. This helps improve operational efficiency and customer satisfaction. The system ensures faster service, better communication, and improved accuracy in order processing. Overall, the Online Food Ordering System enhances the dining experience by providing convenience, speed, and accessibility. It is widely used in modern businesses and plays a significant role in digital transformation in the food industry.

1.2 PROBLEM DEFINITION

Traditional food ordering systems rely heavily on manual processes such as phone calls or in-person visits. These methods are time-consuming, prone to human errors, and inefficient in handling multiple orders simultaneously. Customers often face issues like long waiting times, incorrect orders, and lack of transparency in pricing and availability.

Restaurants also struggle with managing orders effectively, especially during peak hours. Manual record-keeping can lead to misplaced orders, billing errors, and poor customer service. There is also no proper system to track order history or analyze customer preferences. Another major issue is the lack of real-time updates. Customers are unable to track their orders, leading to confusion and dissatisfaction. Additionally, there is limited scope for digital payments in traditional systems, which reduces convenience. Therefore, there is a need for an automated system that can streamline the ordering process, reduce errors, and improve efficiency. The Online Food Ordering System aims to address these challenges by providing a user-friendly and reliable platform for both customers and restaurants.

1.3 OBJECTIVES OF THE PROJECT

The main objective of the Online Food Ordering System is to develop an efficient and user-friendly platform that simplifies the process of ordering food online. The system aims to provide a seamless experience for users by allowing them to browse menus, place orders, and make payments easily.

Another important objective is to reduce manual effort and minimize errors in order processing. By automating the entire system, it ensures accurate order management and faster service delivery. The system also helps restaurants maintain a digital record of orders, which can be used for analysis and decision-making. The project also aims to enhance customer satisfaction by providing features such as order tracking, quick response time, and multiple payment options. It ensures secure transactions and protects user data. Additionally, the system is designed to be scalable and adaptable to different business needs. It can be expanded with additional features like delivery tracking, customer feedback, and promotional offers in the future.

1.4 SCOPE OF THE PROJECT

The scope of the Online Food Ordering System includes the development of a web-based platform that connects customers and restaurants. It covers functionalities such as user registration, login, menu browsing, cart management, and order placement.

The system allows customers to view available food items, add them to the cart, and place orders with ease. It also supports basic payment methods and generates order confirmations. On the admin side, the system enables management of food items, pricing, and order details. However, the current scope is limited to basic features and does not include advanced functionalities like live delivery tracking, GPS integration, or real-time chat support. These features can be added in future enhancements.

The system is mainly designed for small to medium-scale restaurants and can be implemented in local environments. It provides a foundation for further development and integration with mobile applications or third-party services.

1.5 DATABASE MANAGEMENT SYSTEM CONCEPTS

A Database Management System (DBMS) plays a crucial role in the Online Food Ordering System by storing, managing, and retrieving data efficiently. It ensures that all information related to users, orders, and food items is organized and easily accessible. The system uses tables to store data such as customer details, menu items, and order records. Each table consists of rows and columns, where rows represent records and columns represent attributes. Relationships between tables are established using keys such as primary keys and foreign keys. The DBMS ensures data integrity, security, and consistency. It prevents duplication and unauthorized access to sensitive information. Queries are used to retrieve and manipulate data as required by the application.

Additionally, the DBMS supports operations such as insert, update, delete, and select, which are essential for managing the system. It provides a structured approach to handle large volumes of data and improves overall system performance.

CHAPTER 2

LITERATURE SURVEY

The literature survey provides an understanding of existing research and systems related to online food ordering. It helps identify current technologies, methodologies, and challenges in the domain. By analyzing previous systems, we can understand their strengths and weaknesses, which helps in designing a better solution.

Many existing systems have attempted to digitize food ordering using web and mobile platforms. These systems focus on improving customer convenience, reducing manual work, and increasing efficiency. Technologies such as databases, web frameworks, and payment gateways are commonly used. The study of existing systems also highlights issues such as limited scalability, lack of real-time updates, and security concerns. These challenges motivate the development of improved systems with better performance and reliability. Thus, the literature survey plays an important role in understanding the current scenario and designing a system that overcomes existing limitations.

2.1 EXISTING SYSTEM

The existing food ordering system mainly relies on traditional methods such as phone calls or physical visits to restaurants. Some restaurants also use basic web applications that allow customers to view menus and place orders online. In these systems, customers select food items and place orders manually, which are then processed by restaurant staff. The system may include basic features like menu display and order placement, but often lacks advanced functionalities. Most existing systems do not provide real-time updates on order status. Customers are not informed about the progress of their orders, leading to confusion and dissatisfaction. Payment options are also limited, with many systems relying on cash transactions. Additionally, these systems often lack proper database management, making it difficult to store and analyze customer data. This limits the ability of businesses to understand customer preferences

and improve services. Overall, while existing systems provide basic functionality, they are not efficient enough to meet modern user expectations.

2.2 LIMITATIONS OF EXISTING SYSTEM

The existing food ordering systems have several limitations that affect their performance and usability. One of the major drawbacks is the dependency on manual processes, which increases the chances of human errors such as incorrect orders or billing mistakes. Another limitation is the lack of real-time order tracking. Customers are unable to monitor the status of their orders, which leads to uncertainty and poor user experience. This also increases the workload on restaurant staff, as they have to respond to customer queries manually.

Security is also a concern in many existing systems. Without proper authentication and data protection mechanisms, user information may be at risk. Additionally, the absence of secure payment gateways limits the use of online transactions.

Scalability is another issue, as many systems are not designed to handle a large number of users simultaneously. This can lead to system crashes or slow performance during peak hours. These limitations highlight the need for a more advanced and reliable system that can overcome these challenges.

2.3 PROPOSED SYSTEM

The proposed Online Food Ordering System is designed to overcome the limitations of existing systems by providing a fully automated and user-friendly platform. It allows customers to browse menus, select food items, and place orders easily through a web interface. The system includes features such as user registration, secure login, cart management, and online payment options.

Customers can track their orders in real-time, which improves transparency and user satisfaction. On the administrative side, the system provides tools for managing food items, updating prices, and monitoring orders. It also maintains a database of customer and order details, which can be used for analysis and decision-making.

The proposed system uses a structured database and efficient algorithms to ensure fast and accurate processing of orders. It also includes security features such as authentication and data encryption to protect user information. Overall, the proposed system aims to provide a reliable, efficient, and scalable solution for online food ordering.

2.4 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over existing systems. One of the main benefits is improved efficiency, as the entire ordering process is automated. This reduces manual work and minimizes errors. Another advantage is enhanced user experience. Customers can easily browse menus, place orders, and track their status in real-time. The system also provides multiple payment options, making transactions more convenient.

The system ensures better data management by storing all information in a structured database. This allows restaurants to analyze customer preferences and improve their services. It also helps in maintaining accurate records of orders and transactions.

Security is another key advantage, as the system includes authentication mechanisms and secure payment processing. This protects user data and ensures safe transactions. Additionally, the system is scalable and can handle a large number of users without performance issues. It can also be extended with new features in the future.

CHAPTER 3

SYSTEM DESIGN

The system design of the Online Food Ordering System defines the overall architecture and structure of the application. It describes how different components such as users, food items, orders, payments, admin, and delivery interact with each other. The design ensures smooth data flow, efficient processing, and proper database management.

It helps in converting user requirements into a structured system that is easy to implement and maintain. Additionally, the system design provides a clear blueprint for developers to build the application effectively. It helps in identifying system modules, their responsibilities, and interactions. Proper design improves system reliability, scalability, and performance. It also makes future enhancements and maintenance easier.

3.1 ENTITY RELATIONSHIP MODEL

The Entity Relationship (ER) diagram for the Online Food Ordering System represents the structure of the database by showing entities and their relationships. The main entities include User/Customer, Food Item, Order, Payment, Admin, and Delivery. A customer uses the system to browse food items and place orders. Each order contains one or more food items, and each food item can belong to multiple orders. The payment entity is associated with the order to complete the transaction process. The delivery entity handles the shipment of the ordered food to the customer. The admin manages food items and oversees the delivery process. This ER diagram helps in organizing the data efficiently and ensures proper interaction between different components of the system.

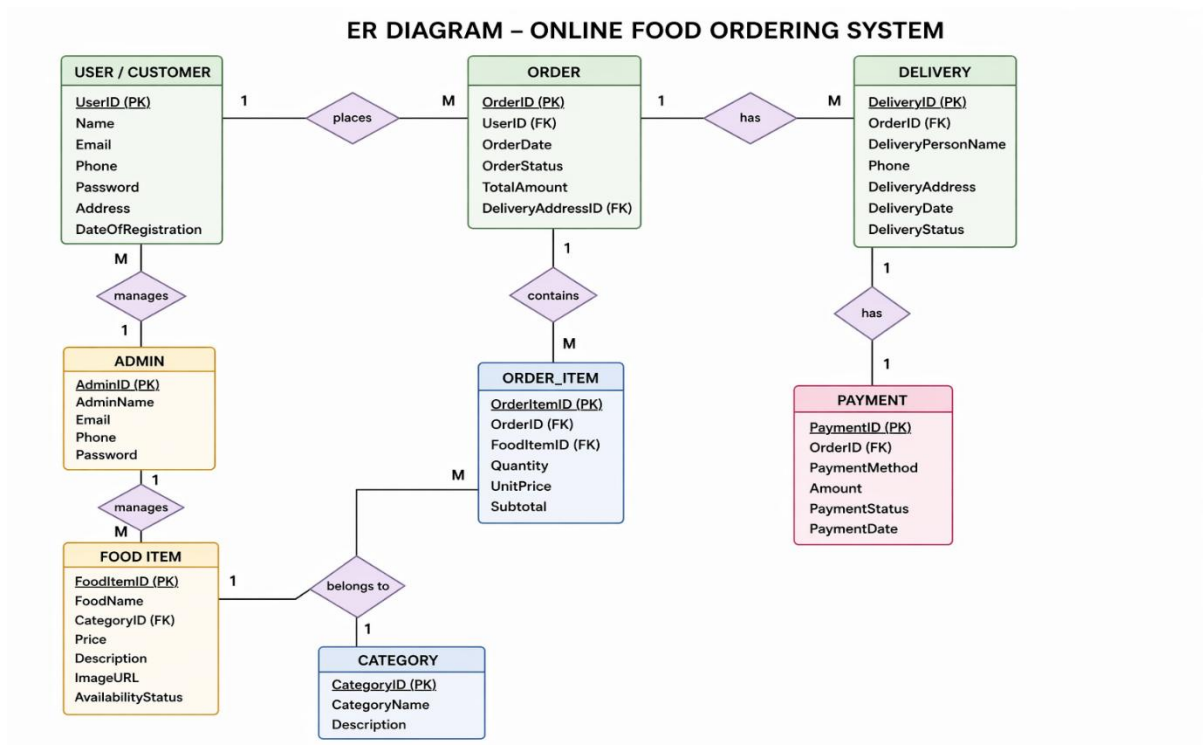


Figure 3.1 Entity Relationship Model

3.2 DATABASE DESIGN

The database design for the Online Food Ordering System is used to store and manage all the data in an organized way. It consists of different tables such as User, Food Item, Order, Payment, Admin, and Delivery. Each table has a primary key to uniquely identify records, and foreign keys are used to create relationships between tables. The User table stores customer details, while the Food Item table contains menu details. The Order table records customer purchases, and the Payment table stores transaction details. The Delivery table manages order delivery information. This structured design reduces data redundancy, maintains data integrity, and allows easy retrieval and management of information in the system.

3.2.1 TABLE STRUCTURE

The Online Food Ordering System database consists of several tables such as User, Food Item, Order, Payment, Admin, and Delivery. Each table is designed to store specific information related to the system. These tables are interconnected using relationships to ensure proper data flow and organization. This structure helps in efficient storage, retrieval, and management of data.

3.2.2 ATTRIBUTES

Each table contains attributes (fields) that describe the data stored in it.

For example:

- User → User_ID, Name, Email, Password, Phone_Number, Address
- Food Item → Food_Item_ID, Name, Description, Price, Category
- Order → Order_ID, User_ID, Order_Date, Quantity, Total_Amount
- Payment → Payment_ID, Order_ID, Payment_Method, Amount, Status
- Delivery → Delivery_ID, Order_ID, Delivery_Status, Delivery_Date
- Admin → Admin_ID, Name, Email, Password, Role

These attributes define the properties of each entity in the system.

3.2.3 KEYS (PRIMARY AND FOREIGN KEY)

Keys are used to uniquely identify records and establish relationships between tables.

- **Primary Key:** A unique identifier for each record in a table (e.g., User_ID, Food_Item_ID).
- **Foreign Key:** A field that links one table to another (e.g., User_ID in Order table, Order_ID in Payment and Delivery tables).

Primary and foreign keys ensure data integrity and help in maintaining relationships between different tables.

CHAPTER 4

PROJECT METHODOLOGY

Project Methodology defines the step-by-step process followed to develop the Online Food Ordering System. It ensures that the project is completed in a systematic and organized manner. The development of this system follows a structured approach similar to the Software Development Life Cycle (SDLC). The stages include requirement analysis, system design, implementation, testing, and deployment. In the requirement analysis phase, the needs of users and system functionalities are identified. In the design phase, the architecture, database, and user interface are planned. During implementation, the system is developed using programming languages and database tools. Testing is performed to identify and fix errors, ensuring the system works correctly. Finally, the system is deployed for users and maintained for future improvements.

4.1 WORKFLOW OF THE SYSTEM

The workflow of the Online Food Ordering System begins with the user registering or logging into the system using valid credentials. After logging in, the user can browse the menu and view various food items along with their details such as price and description. The user then selects the desired items and adds them to the cart. Once the selection is complete, the user proceeds to place the order by confirming the details. The system then processes the payment through available methods. After successful payment, an order confirmation is generated and the details are stored in the database. Finally, the admin receives the order information and processes it for preparation and delivery..

4.2 BLOCK DIAGRAM

The block diagram of the Online Food Ordering System shows how different parts of the system work together. The user interacts with the system through the user interface, where they can log in, browse the menu, and place orders. These requests are sent to the application server, which processes them using business logic. The server communicates with the database to store and retrieve data such as user details, food items, and orders. The processed result is then sent back to the user. At the same time, the admin panel allows the administrator to manage food items and view orders. This structure ensures smooth communication between all components and efficient system operation.

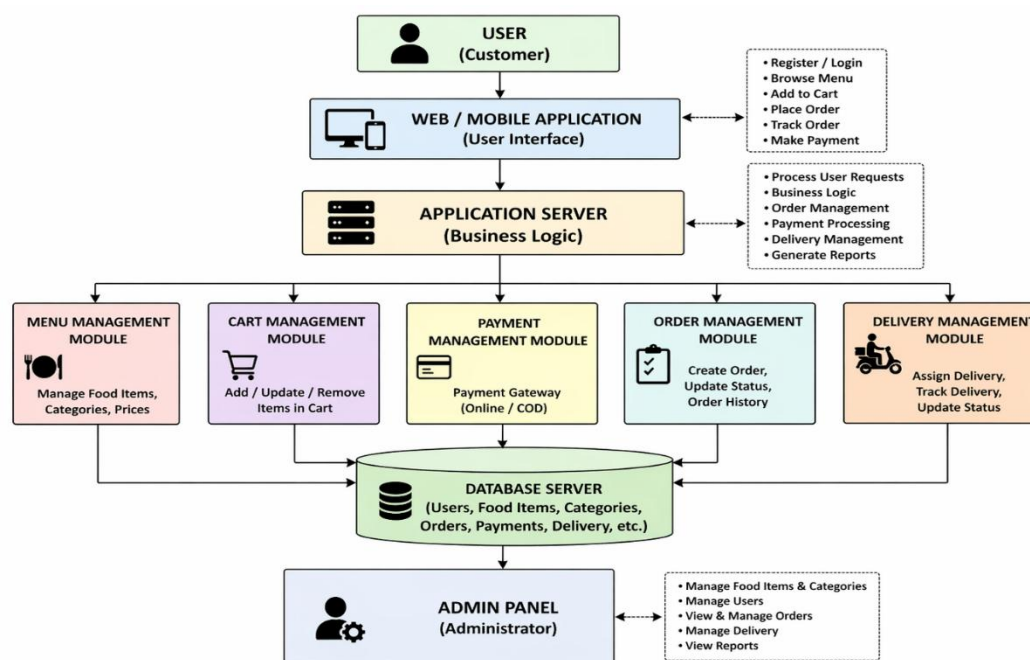


Fig 4.1 Online Food Ordering System

CHAPTER 5

SYSTEM REQUIREMENTS

System Requirements specify the resources needed to develop and run the Online Food Ordering System. These requirements are divided into hardware and software components. Proper system requirements ensure smooth functioning, better performance, and compatibility. They help in identifying the necessary tools and technologies required for development and execution. The system is designed to run on basic computer configurations, making it accessible and cost-effective. Both hardware and software requirements are chosen to support efficient development and user interaction.

5.1 HARDWARE REQUIREMENTS

The hardware requirements define the physical components needed to run the system. The Online Food Ordering System does not require high-end hardware and can function on standard systems.

Minimum Requirements:

- Processor: Intel Core i3 or above
- RAM: 4 GB or higher
- Hard Disk: 500 GB
- Monitor: Standard display
- Input Devices: Keyboard and Mouse

These requirements ensure that the system runs smoothly without performance issues. Higher configurations can improve speed and efficiency, especially when handling multiple users.

5.2 SOFTWARE REQUIREMENTS

The software requirements include the tools and technologies used to develop and run the system.

Development Tools:

- Programming Language: HTML, CSS, JavaScript
- Backend: PHP / Python / Java (any one)
- Database: MySQL

Operating System:

- Windows / Linux

Web Browser:

- Google Chrome / Mozilla Firefox

Other Tools:

- XAMPP / WAMP Server (for local server environment)

These software components help in building a user-friendly interface, managing data efficiently, and ensuring smooth communication between different parts of the system.

CHAPTER 6

MODULE DESCRIPTION

The Online Food Ordering System is divided into different modules to simplify development and improve maintainability. Each module performs a specific function and works together to complete the system. Modular design makes the system more organized, easy to understand, and easier to update or expand in the future.

6.1 USER MODULE

The User Module handles all activities related to the customer. It allows users to interact with the system easily.

Functions:

- User registration and login
- Browse food menu
- Add items to cart
- Place orders
- View order history

6.2 ADMIN MODULE

The Admin Module is responsible for managing the system. It provides control over all operations.

Functions:

- Add, update, and delete food items
- Manage user accounts
- View and process orders
- Generate reports

6.3 DATABASE MODULE

The Database Module manages all data in the system. It stores and retrieves information as required.

Functions:

- Store user details
- Maintain food menu data
- Record order and payment details
- Ensure data consistency

6.4 AUTHENTICATION MODULE

The Authentication Module ensures system security by verifying user identity.

Functions:

- User login validation
- Password protection
- Prevent unauthorized access

This module protects sensitive information and ensures that only authorized users can access the system.

6.5 REPORT GENERATION MODULE

This module is used to generate reports based on system data.

Functions:

- Generate order reports
- Analyze sales data
- View user activity

Reports help administrators understand system performance and make better decisions.

CHAPTER 7

IMPLEMENTATION

Implementation is the stage where the Online Food Ordering System is developed into a working application. The system is designed based on the planned structure and requirements. Frontend technologies like HTML, CSS, and JavaScript are used for the user interface. Backend technologies such as Python or PHP handle the system operations. MySQL database is used to store and manage all the data. All modules like user, admin, order, and payment are developed and connected. The system is tested to find and fix errors. It ensures proper working of registration, food ordering, payment, security, and performance.

7.1 DATABASE CREATION

```
CREATE DATABASE FoodOrderingSystem;
```

```
USE FoodOrderingSystem;
```

```
CREATE TABLE User (
```

```
    User_ID INT PRIMARY KEY,
```

```
    Name VARCHAR(50),
```

```
    Email VARCHAR(50),
```

```
    Password VARCHAR(50)
```

```
);
```

```
CREATE TABLE Food (
```

```
    Food_ID INT PRIMARY KEY,
```

```
    Food_Name VARCHAR(50),
```

```
    Price DECIMAL(10,2),
```

```
    Category VARCHAR(50)
```

);

CREATE TABLE Orders (

Order_ID INT PRIMARY KEY,

User_ID INT,

Total_Amount DECIMAL(10,2),

FOREIGN KEY (User_ID) REFERENCES User(User_ID)

);

7.2 SQL QUERIES

SQL queries are used to perform operations on the database such as inserting, retrieving, updating, and deleting data.

7.2.1 INSERT

INSERT INTO User VALUES (1, 'Shobana', 'shobana@gmail.com', '1234');

INSERT INTO Food VALUES (101, 'Pizza', 250.00, 'Fast Food');

7.2.2 SELECT

SELECT * FROM User;

SELECT Food_Name, Price FROM Food;

7.2.3 UPDATE

UPDATE User

SET Name = 'Shobana M'

WHERE User_ID = 1;

7.2.4 DELETE

Used to remove records from the table.

```
DELETE FROM Food
```

```
WHERE Food_ID = 101;
```

7.3 CONSTRAINTS

Constraints are rules applied to database tables to ensure data integrity and accuracy.

7.3.1 PRIMARY KEY

- Uniquely identifies each record
- Cannot be NULL

Example:

```
User_ID INT PRIMARY KEY
```

7.3.2 FOREIGN KEY

- Links two tables
- Maintains relationship

Example:

```
FOREIGN KEY (User_ID) REFERENCES User(User_ID)
```

7.3.3 NOT NULL

- Ensures a column cannot have NULL values

```
Name VARCHAR(50) NOT NULL
```

7.3.4 UNIQUE

- Ensures all values in a column are different

Example: Email VARCHAR(50) UNIQUE

CONCLUSION

The Online Food Ordering System has been successfully designed and developed to simplify and modernize the process of ordering food. This system provides an efficient and user-friendly platform for customers to browse menus, place orders, and make payments conveniently from their devices.

The project successfully eliminates many drawbacks of traditional food ordering methods such as manual errors, long waiting times, and lack of proper order management. By automating the entire process, the system ensures accuracy, speed, and improved customer satisfaction.

Through the implementation of various modules such as user management, admin control, authentication, and database handling, the system demonstrates effective integration of frontend and backend technologies. The use of a structured database ensures proper data storage, retrieval, and security.

Additionally, the system is scalable and can be enhanced in the future by adding advanced features like real-time delivery tracking, mobile application support, and online payment gateways. These improvements can further increase efficiency and user convenience.

In conclusion, the Online Food Ordering System is a reliable and practical solution that meets the current needs of digital food services. It enhances both customer experience and business operations, making it a valuable system in today's technology-driven world.

APPENDIX A: SOURCE CODE STRUCTURE

MYSQL DATABASE

```
CREATE DATABASE IF NOT EXISTS food_ordering_system;

USE food_ordering_system;

-- TABLES

CREATE TABLE IF NOT EXISTS categories (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100) NOT NULL UNIQUE,

    description TEXT,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE IF NOT EXISTS food_items (

    id INT AUTO_INCREMENT PRIMARY KEY,

    category_id INT,

    name VARCHAR(150) NOT NULL,

    description TEXT,

    price DECIMAL(10, 2) NOT NULL,

    image_url VARCHAR(255),

    is_available TINYINT(1) DEFAULT 1,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET

    NULL

);
```

-- SAFE INSERTS (no duplicate errors)

INSERT IGNORE INTO categories (name, description) VALUES

('Starters', 'Delicious appetizers to kick off your meal'),

('Main Course', 'Hearty and satisfying main dishes'),

('Biryani & Rice', 'Flavourful rice dishes'),

('Desserts', 'Sweet treats to end your meal'),

('Beverages', 'Refreshing drinks');

INSERT IGNORE INTO food_items (category_id, name, description, price, is_available) VALUES

(1, 'Chicken 65', 'Spicy deep-fried chicken with curry leaves', 120.00, 1),

(2, 'Paneer Butter Masala', 'Rich paneer in buttery tomato gravy', 160.00, 1),

(3, 'Veg Biryani', 'Fragrant basmati rice with mixed vegetables', 140.00, 1),

(4, 'Ice Cream', 'Creamy vanilla / chocolate / strawberry', 70.00, 1),

(5, 'Fresh Lime Soda', 'Chilled lime soda, sweet or salted', 40.00, 1);

-- ADMIN FIX

INSERT INTO admins (username, password)

VALUES ('admin', 'admin123')

ON DUPLICATE KEY UPDATE password = 'admin123';

Python code:

"""

Food Ordering System - Flask Backend

Run: python app.py

Then open: <http://127.0.0.1:5000>

```
"""
```

```
from flask import Flask, render_template, request, redirect, url_for, session, jsonify,
flash
```

```
import mysql.connector
```

```
import json
```

```
app = Flask(__name__)
```

```
app.secret_key = "food_order_secret_2024"
```

```
DB_CONFIG = {
```

```
    "host":    "localhost",
```

```
    "user":    "root",
```

```
    "password": "shso@30", # ← change this
```

```
    "database": "food_ordering_system"
```

```
}
```

```
def get_db():
```

```
    return mysql.connector.connect(**DB_CONFIG)
```

```
@app.route("/")
```

```
def index():
```

```
    db = get_db()
```

```
    cur = db.cursor(dictionary=True)
```

```
    cur.execute("SELECT * FROM categories")
```

```
    categories = cur.fetchall()
```

```
cur.execute("SELECT f.*, c.name AS category_name FROM food_items f JOIN
categories c ON f.category_id = c.id WHERE f.is_available = 1")
```

```
items = cur.fetchall()
```

```
db.close()
```

```
return render_template("index.html", categories=categories, items=items)
```

```
@app.route("/menu")
```

```
def menu():
```

```
    db = get_db()
```

```
    cur = db.cursor(dictionary=True)
```

```
    cat_id = request.args.get("category", None)
```

```
    cur.execute("SELECT * FROM categories")
```

```
    categories = cur.fetchall()
```

```
    items = cur.fetchall()
```

```
    db.close()
```

```
    return render_template("menu.html", categories=categories, items=items,
selected_cat=cat_id)
```

```
    if request.method == "POST":
```

```
        name = request.form["name"]
```

```
        email = request.form["email"]
```

```
        phone = request.form["phone"]
```

```
        address = request.form["address"]
```

```
        payment = request.form.get("payment", "Cash")
```

```
        if existing:
```



```

        customer_id = existing["id"]

    else:

        cur.execute("INSERT INTO customers (name, email, phone, address)
VALUES (%s,%s,%s,%s)",

                (name, email, phone, address))

    db.commit()

    order_id = cur.lastrowid

    for item in details:

        cur.execute("INSERT INTO order_items (order_id, food_item_id, quantity,
unit_price) VALUES (%s,%s,%s,%s)",

                (order_id, item["id"], item["qty"], item["price"]))

    db.close()

    return render_template("checkout.html", cart_items=details, total=total)

@app.route("/order_history", methods=["GET", "POST"])
def order_history():

    orders = []

    email = ""

    if request.method == "POST":

        email = request.form.get("email", "")

        db = get_db()

        cur = db.cursor(dictionary=True)

        cur.execute("SELECT o.* FROM orders o JOIN customers c ON
o.customer_id=c.id WHERE c.email=%s ORDER BY o.ordered_at DESC", (email,))

        orders = cur.fetchall()

```

```
db.close()
```

```
return render_template("order_history.html", orders=orders, email=email)
```

Index.html

```
{% extends "base.html" %}
```

```
{% block title %}SpiceBox — Home{% endblock %}
```

```
{% block content %}
```

```
<section class="hero">
```

```
<div class="hero-content">
```

```
<p class="hero-tag">🍲 Madurai's Favourite</p>
```

```
<h1>Fresh Food,<br><span>Fast Delivery</span></h1>
```

```
<p class="hero-sub">Authentic flavours delivered hot to your doorstep.<br>Order  
in minutes, eat in joy.</p>
```

```
<a href="{{ url_for('menu') }}" class="btn-primary">Explore Menu →</a>
```

```
</div>
```

```
<section class="featured-section">
```

```
<h2 class="section-title">Popular Items</h2>
```

```
<div class="food-info">
```

```
<span class="food-cat">{{ item.category_name }}</span>
```

```
<h3>{{ item.name }}</h3>
```

```
<p class="food-desc">{{ item.description }}</p>
```

```
<div class="food-footer">
```

```
<span class="price">₹{{ "%.0f"|format(item.price) }}</span>
```

```
<a href="{{ url_for('add_to_cart', item_id=item.id) }}" class="btn-add">+
```

```
</section>
```

```
{% endblock %}
```

Menu.html

```
{% extends "base.html" %}
```

```
{% block title %}Menu — SpiceBox{% endblock %}
```

```
{% block content %}
```

```
<div class="page-header">
```

```
  <h1>Our Menu</h1>
```

```
  <p>Fresh, authentic, made to order</p>
```

```
</div>
```

```
<div class="menu-layout">
```

```
  <aside class="filter-sidebar">
```

```
    <h3>Categories</h3>
```

```
    <a href="{ { url_for('menu') } }" class="filter-link {% if not selected_cat
{%} active {% endif %} ">All Items</a>
```

```
    {% for cat in categories %}
```

```
      <a href="{ { url_for('menu', category=cat.id) } }"
```

```
        class="filter-link {% if selected_cat == cat.id|string %} active {% endif %} ">
```

```
        { { cat.name } }
```

```
      </a>
```

```
    {% endfor %}
```

```
</aside>
```

```
<div class="food-grid">
```

```

    {% for item in items %}

    <div class="food-card">

    </div>

    {% else %}

    <div class="empty-state">

    <span>🍲</span>

    <p>No items found in this category.</p>

    </div>

    {% endfor %}

</div>

</div>

{% endblock %}

```

Cart.html

```

{% extends "base.html" %}

{% block title %}Cart — SpiceBox{% endblock %}

{% block content %}

<div class="page-header">

  <h1>Your Cart</h1>

</div>

{% if cart_items %}

<div class="cart-layout">

  <div class="cart-items">

    {% for item in cart_items %}

```

```

<div class="cart-row">

  <span class="cart-emoji">

    {% if 'Biryani' in item.name %} 🍛

    {% elif 'Chicken' in item.name %} 🍗

    {% elif 'Paneer' in item.name %} 🧀

    {% else %} 🍲 {% endif %}

  </span>

  <div class="cart-item-info">

    <h4>{{ item.name }}</h4>

    <span>₹{{ "%.0f"|format(item.price) }} each</span>

    {% endfor %}

  </div>

<div class="cart-summary">

  <h3>Order Summary</h3>

  <div class="summary-row"><span>Subtotal</span><span>₹{{
  "%.0f"|format(total) }}</span></div>

  {% endif %}

{% endblock %}

```

Checkout.html

```

{% extends "base.html" %}

{% block title %}Checkout — SpiceBox{% endblock %}

{% block content %}

<div class="page-header">

```

```

<h1>Checkout</h1>

</div>

<div class="checkout-layout">

  <form method="POST" class="checkout-form">

    <h3>Your Details</h3>

    </div>

    <div class="form-group full">

      <label>Delivery Address *</label>

      <textarea name="address" required rows="3" placeholder="House no, Street, City,
Pincode"></textarea>

      <button type="submit" class="btn-primary full-width">Place Order 🛒</button>

    </form>

    <div class="cart-summary">

      <h3>Your Order</h3>

      {% for item in cart_items %}

      <div class="summary-row">

        <span>{{ item.name }} × {{ item.qty }}</span>

        <span>₹{{ "%.0f"|format(item.subtotal) }}</span>

      </div>

      {% endfor %}

      <hr/>

      <div class="summary-row total"><span>Total</span><span>₹{{
"%.0f"|format(total) }}</span></div>

```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

Orderconfirmattion.html

```
{% extends "base.html" %}
```

```
{% block title %}Order Confirmed — SpiceBox{% endblock %}
```

```
{% block content %}
```

```
<div class="confirmation-page">
```

```
<div class="confirm-icon">  </div>
```

```
<h1>Order Placed!</h1>
```

```
<p class="confirm-sub">Thank you, <strong>{{ order.customer_name }}</strong>!  
Your order #<strong>{{ order.id }}</strong> has been received.</p>
```

```
<div class="confirm-card">
```

```
<div class="confirm-row"><span>Order ID</span><span>#{{ order.id  
}}</span></div>
```

```
<div class="confirm-row"><span>Status</span><span class="badge badge-  
pending">{{ order.status }}</span></div>
```

```
<div class="confirm-row"><span>Payment</span><span>{{  
order.payment_method }}</span></div>
```

```
<div class="confirm-row"><span>Total</span><span>₹{{  
"%0.0f"|format(order.total_amount) }}</span></div>
```

```
<hr/>
```

```
<h4>Items Ordered</h4>
```

```
<div class="confirm-actions">
```

```

    <a href="{{ url_for('menu') }}" class="btn-primary">Order More →</a>

    <a href="{{ url_for('order_history') }}" class="btn-secondary">View My
Orders</a>

</div>

</div>

{% endblock %}

```

Orderhistory.html

```

{% extends "base.html" %}

{% block title %}My Orders — SpiceBox{% endblock %}

{% block content %}

<div class="page-header">

    <h1>Order History</h1>

    <p>Track all your past orders</p>

</div>

<div class="history-lookup">

    <tbody>

        {% for order in orders %}

            {% endfor %}

    </tbody>

</table>

</div>

{% elif email %}

<div class="empty-state">

```



```
<span> 🇵🇷 </span>
```

```
<p>No orders found for <strong>{{ email }}</strong></p>
```

```
</div>
```

```
{% endif %}
```

```
{% endblock %}
```

Admin.login

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8"/>
```

```
<meta name="viewport" content="width=device-width,initial-scale=1"/>
```

```
<title>Admin Login — SpiceBox</title>
```

```
<link
```

```
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;600&display=s
wap" rel="stylesheet">
```

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}" />
```

```
</head>
```

```
<body class="admin-body">
```

```
<div class="admin-login-box">
```

```
<div class="admin-logo"> 🍷 </div>
```

```
<h2>Admin Login</h2>
```

```
<p>SpiceBox Management Panel</p>
```

```
{% with messages = get_flashed_messages(with_categories=true) %}
```

```

{% for cat, msg in messages %}

    <div class="flash flash-{{ cat }}">{{ msg }}</div>

{% endfor %}

{% endwith %}

<form method="POST">

    <div class="form-group">

        <label>Username</label>

        <input type="text" name="username" required placeholder="admin" autofocus/>

    </div>

</form>

</body>

</html>

```

Admin.dashboard

```

{% extends "admin_base.html" %}

{% block title %}Dashboard — Admin{% endblock %}

{% block content %}

    <h2 class="admin-page-title">Dashboard</h2>

    <div class="stats-grid">

        <div class="stat-card">

            <span class="stat-icon"> 📦 </span>

            <div><p class="stat-label">Total Orders</p><h3>{{ total_orders }}</h3></div>

        </div>

        <div class="stat-card">

            <span class="stat-icon"> 💰 </span>


```

```
<div><p class="stat-label">Revenue (Delivered)</p><h3>₹{{
"%0.0f"|format(revenue) }}</h3></div>
```

```
</div>
```

```
<h3 class="admin-section-title">Recent Orders</h3>
```

```
<div class="table-wrap">
```

```
<table class="admin-table">
```

```
<tbody>
```

```
{% for o in recent_orders %}
```

```
{% endfor %}
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
{% endblock %}
```

Admindashboard.html

```
{% extends "admin_base.html" %}
```

```
{% block title %}Dashboard — Admin{% endblock %}
```

```
{% block content %}
```

```
<h2 class="admin-page-title">Dashboard</h2>
```

```
<div class="stats-grid">
```

```
<div class="stat-card">
```

```
<span class="stat-icon"> 📦 </span>
```

```
<div><p class="stat-label">Total Orders</p><h3>{{ total_orders }}</h3></div>
```

```
</div>
```

```

<div class="stat-card">

    <span class="stat-icon"> 💰 </span>

    <div><p class="stat-label">Revenue (Delivered)</p><h3>₹ {{
"%0.0f"|format(revenue) }}</h3></div>

</div>

<h3 class="admin-section-title">Recent Orders</h3>

<div class="table-wrap">

    <table class="admin-table">

        <thead>
<tr><th>ID</th><th>Customer</th><th>Amount</th><th>Status</th><th>Time</th>
></tr>

        </thead>

        <tbody>

            {% for o in recent_orders %}

                {% endfor %}

        </tbody>

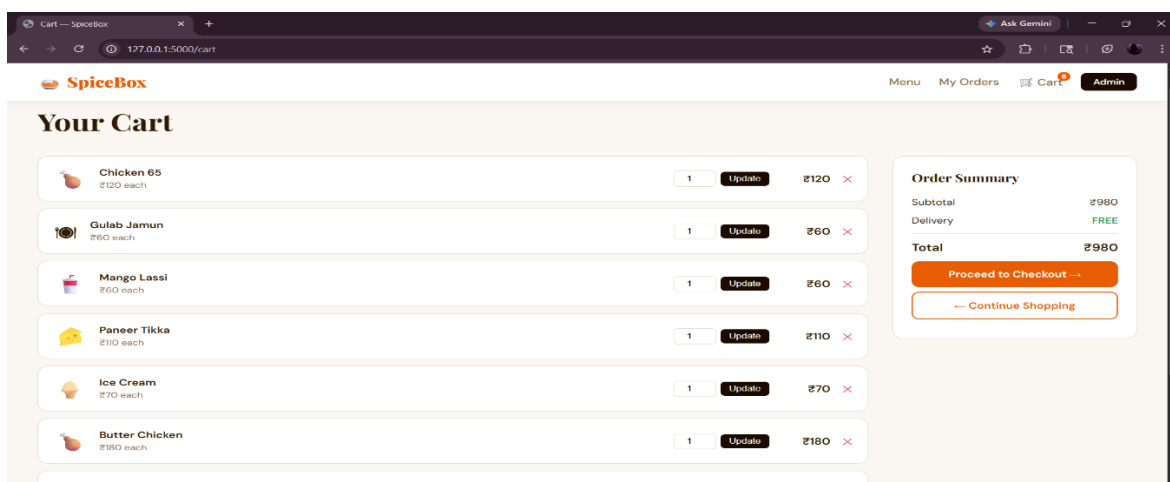
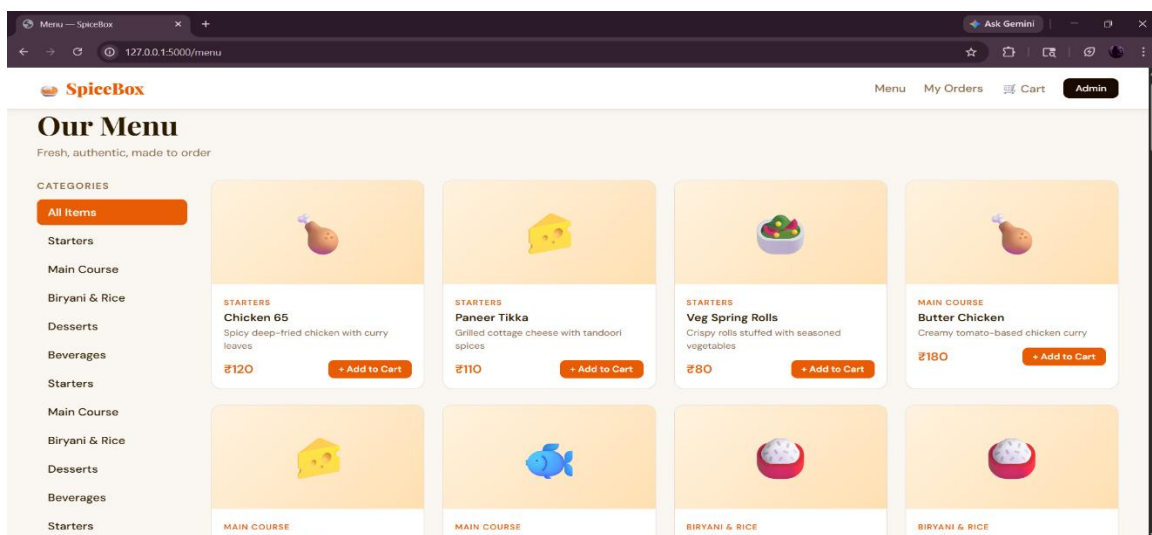
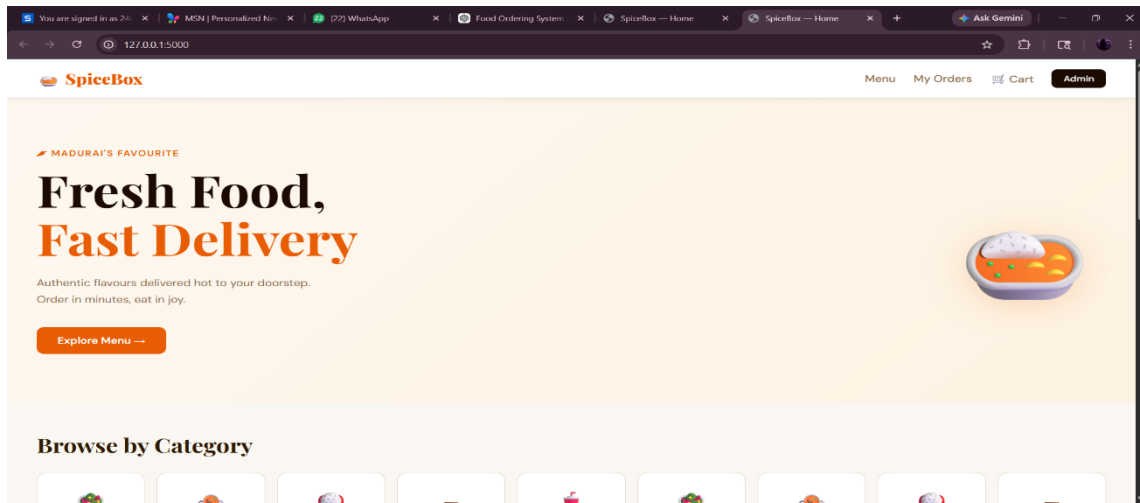
    </table>

</div>

{% endblock %}

```

APPENDIX B: SCREENSHOTS



Admin Login — SpiceBox

127.0.0.1:5000/admin/login



Admin Login
SpiceBox Management Panel

USERNAME

PASSWORD

Login

[← Back to Site](#)

Dashboard — Admin

127.0.0.1:5000/admin

SpiceBox Admin

[Dashboard](#)
[Orders](#)
[Food Items](#)
[View Site](#)
[Logout](#)

Dashboard

TOTAL ORDERS
6

REVENUE (DELIVERED)
₹0

FOOD ITEMS
52

CUSTOMERS
5

Recent Orders

ID	CUSTOMER	AMOUNT	STATUS	TIME
#6	abinaya	₹310	Pending	22 Apr, 04:27 PM
#5	sakthivel	₹300	Pending	22 Apr, 04:10 PM
#4	aari	₹230	Pending	22 Apr, 04:08 PM
#3	aari	₹230	Pending	20 Apr, 02:37 PM
#2	tftghjbnb	₹310	Pending	13 Apr, 10:52 PM
#1	shob	₹320	Pending	13 Apr, 10:35 PM

Orders — Admin

My Orders — SpiceBox

127.0.0.1:5000/order_history

 **SpiceBox**

[Menu](#) [My Orders](#) [Cart](#) [Admin](#)

Order History

Track all your past orders

[Search Orders](#)

Order ID	Date & Time	Total	Payment	Status
#4	22 Apr 2026, 04:08 PM	₹230	Cash	Pending
#3	20 Apr 2026, 02:37 PM	₹230	Cash	Pending

© 2024 SpiceBox — Food Ordering System | Built with Python + Flask + MySQL

Orders -- Admin

Checkout -- SpiceBox

+

Ask Gemini

127.0.0.1:5000/checkout

☆

MenuMy OrdersCartAdmin

Checkout

Your Details

FULL NAME *

e.g. Ravi Kumar

EMAIL *

you@example.com

PHONE *

+91 9876543210

PAYMENT METHOD

Cash on Delivery

DELIVERY ADDRESS *

House no, Street, City, Pincode

SPECIAL INSTRUCTIONS

e.g. Extra spicy, no onions...

Place Order

Your Order

Chicken 65 × 1	₹120
Gulab Jamun × 1	₹60
Mango Lassi × 1	₹60
Paneer Tikka × 1	₹110
Ice Cream × 1	₹70
Butter Chicken × 1	₹180
Fish Curry × 1	₹200
Chicken Biryani × 1	₹180
Total	₹980

Orders -- Admin

Order Confirmed -- SpiceBox

+

Ask Gemini

127.0.0.1:5000/order_confirmation/

☆

MenuMy OrdersCartAdmin

Order Placed!

Thank you, shyam! Your order #7 has been received.

Order ID

#7

Status

Pending

Payment

Cash

Total

₹980

Items Ordered

Chicken 65 × 1

₹120

Gulab Jamun × 1

₹60

Mango Lassi × 1

₹60

Paneer Tikka × 1

₹110

Ice Cream × 1

₹70

Butter Chicken × 1

₹180

Fish Curry × 1

₹200

Chicken Biryani × 1

₹180

Order More

View My Orders

REFERENCES

1. Python Software Foundation 2024, Python Documentation, viewed online, <https://docs.python.org/3/>.
2. Flask Documentation 2024, Flask Web Framework Guide, Pallets Projects, viewed online, <https://flask.palletsprojects.com>.
3. Apache Friends 2024, XAMPP Installation and User Guide, Apache Friends, viewed online, <https://www.apachefriends.org>.
4. Mozilla Developer Network (MDN) 2024, HTML, CSS and JavaScript Guide, viewed online, <https://developer.mozilla.org>.
5. W3Schools 2024, HTML, CSS, JavaScript, and PHP Tutorials, viewed online, <https://www.w3schools.com>.
6. MySQL 2024, MySQL Reference Manual, Oracle Corporation, viewed online, <https://dev.mysql.com/doc/>.
7. Oracle 2023, Database Concepts and SQL Guide, Oracle Documentation, viewed online, <https://www.oracle.com/database>.
8. GeeksforGeeks 2024, Online Food Ordering System Project, viewed online, <https://www.geeksforgeeks.org>.
9. TutorialsPoint 2024, Web Development and DBMS Tutorials, viewed online, <https://www.tutorialspoint.com>.
10. Stack Overflow 2024, Programming Discussions and Developer Solutions, viewed online, <https://stackoverflow.com>.